

Mechanically Verified Optimization and UPPAAL Export for Timed Büchi Automata

ELIE FARES¹, JEAN-PAUL BODEVEIX², HUSSAIN AL-AQRABI¹ AND AZAR SALAMI¹

¹Higher Colleges of Technology, Sharjah, United Arab Emirates (e-mail: efares@hct.ac.ae, halaqrabi@hct.ac.ae, asalami@hct.ac.ae)

²IRIT, UPS, Université de Toulouse, France (e-mail: bodeveix@irit.fr)

ABSTRACT Timed Büchi automata (TBAs) produced by translators or supplied by other tools can contain redundant timing constraints, resets, transitions, clocks, and states, and may use disjunctive invariants that are unsuitable for direct UPPAAL export. We present a Coq-verified pipeline that synthesizes location invariants, propagates timing bounds, removes infeasible transitions and dead resets, merges synchronously reset clocks and equivalent transitions and locations, and converts the result to conjunctive guards and invariants. Every pass preserves the timed-word language of every input TBA under the development's existential initial-clock semantics. In particular, the optimization theorem is generic in the automaton and in the number of rounds; the export theorem composes the verified transformations into a UPPAAL-oriented representation. The transformations are extracted to OCaml and used in `mtl2tba`. We report measurements on 31 automata generated from a fixed MTL benchmark, plus UPPAAL case studies. The data show large transition reductions on several inputs, while export can increase the edge count when it removes disjunctive invariants. These experiments evaluate the end-to-end tool on the benchmark automata; they do not measure a broad corpus of arbitrary TBAs.

INDEX TERMS Coq, formal verification, program extraction, timed automata, timed Büchi automata, UPPAAL, verified optimization.

I. INTRODUCTION

Timed Büchi automata are a common intermediate representation for real-time requirements and are also used as inputs to timed-language analysis tools. An automaton that is correct at the language level may still be inconvenient for downstream verification: its transition guards can repeat constraints, some resets may never affect a future guard, multiple clocks may evolve in lockstep, and the same behavior may be represented by many transitions. In addition, merging transitions can produce disjunctive guards and invariants, while UPPAAL-oriented automata are most useful with conjunctive guards and conjunctive upper-bound invariants.

An optimization pass is safe only when it preserves accepted timed words, including the treatment of clock resets, event labels, and Büchi acceptance. Several passes also interact. Propagating a lower bound may make a transition contradictory, but a synthetic nonnegativity guard can keep an otherwise dead clock live unless guards are normalized before reset analysis. Splitting a disjunctive invariant into copies also requires a condition on entry: choosing a copy whose invariant admits less time than another copy can change the accepted language.

This paper presents a mechanically verified post-processing and export pipeline for TBAs. Its central results quantify over an arbitrary input TBA, rather than only automata generated from a particular temporal logic. In Coq, we

prove that each optimization and export operation preserves the automaton's timed-word language under the acceptance semantics formalized in the development. The optimizer theorem holds for every finite number of rounds, so the implementation can stop after reaching a structural fixed point or after its configured iteration bound without relying on convergence for correctness.

The transformations have two stages. The optimization stage synthesizes upper-bound location invariants, propagates target invariants backward into guards, propagates entry lower bounds forward, removes contradictory and unreachable transitions, removes dead resets, merges clocks reset synchronously, tightens guards, and merges transitions and locations. The export stage merges transitions with equal source, target, label, and resets; eliminates disjunctive invariants by creating copies with conjunctive invariants and clock-difference entry conditions; splits disjunctive guards; normalizes the result; and emits symbolic transitions for drawing.

The verified transformations are extracted to OCaml and used by `mtl2tba`. That tool includes an upstream MTL-to-TBA construction and an interface to Spot, but the preservation theorems in this paper apply to every TBA passed to the verified post-processing functions. The experiments use automata from 31 fixed MTL benchmark configurations, not a diverse arbitrary-TBA corpus. We therefore distinguish the

universal proof claims from the benchmark-specific performance measurements.

The contributions are:

- a Coq development of language-preserving TBA transformations, including invariant synthesis and propagation, clock and state simplification, transition merging, and disjunctive-invariant elimination;
- a generic theorem for every TBA and every number of optimization rounds, plus a verified export composition whose final guards and invariants are conjunctive;
- an extracted implementation and an explicit account of its interface and trust boundaries; and
- a reproducible evaluation on the tracked 31-configuration benchmark and four system-model case studies, with results and limitations reported from repository data.

This paper treats the TBA optimization/export layer. Correctness of any upstream logic-to-automaton translation remains a separate contract. Our proofs do not assume that an input automaton came from MTL, Spot, or this tool.

II. TIMED AUTOMATA AND THE PRESERVATION CONTRACT

A. TIMED WORDS AND TBAS

A timed word is an infinite sequence $w = (a_i, t_i)_{i \in \mathbb{N}}$, where a_i is an event from a finite alphabet Σ , timestamps are strictly increasing, and time diverges: $\lim_{i \rightarrow \infty} t_i = +\infty$. Let $t_{-1} = 0$ and $\delta_i = t_i - t_{i-1}$. A clock valuation maps a finite clock set X to $\mathbb{R}_{\geq 0}$. Guards are finite conjunctions of event literals and clock comparisons $x \bowtie c$ for $\bowtie \in \{<, \leq, =, \geq, >\}$; transitions also carry a set of clocks to reset.

A TBA is a tuple $\mathcal{A} = (L, \ell_0, X, E, Acc)$, where L is a finite location set, ℓ_0 is the initial location, X is the clock set, E is a finite set of transitions $(\ell, \alpha, g, Z, \ell')$, α is an event condition, g is a clock guard, $Z \subseteq X$ is a reset set, and $Acc \subseteq L$ is the set of Büchi-accepting locations. The starting automata in the verified optimization are represented without location invariants; synthesis adds them as a separate annotation. Exported invariants are conjunctions of upper bounds.

The Coq development interprets transitions over an extended timed word that records clock values and reset decisions. For each clock x , values remain nonnegative and evolve according to

$$v_{i+1}(x) = \begin{cases} \delta_i, & x \in Z_i, \\ v_i(x) + \delta_i, & x \notin Z_i. \end{cases}$$

A run chooses one enabled transition per position and visits accepting locations infinitely often. The accepted language existentially quantifies over clock-consistent extensions of the base timed word, including the initial clock values. This is the acceptance notion of the generic preservation theorems; conventional initialization at zero is a separate deployment concern.

For an annotated automaton, an invariant must hold throughout the delay spent at its location. Since synthesized

invariants are conjunctions of non-strict upper bounds and clocks are nondecreasing during a delay, holding at the event at the end of the delay is equivalent to holding throughout that stay. The proof formalizes this trace-model equivalence and uses it to show that the annotations and strengthening operations preserve timed words.

B. WHAT EXPORT MUST PRESERVE

The export must retain the same transition alphabet, Büchi marking, clock meaning, and timed-word language. It may change the number of locations, transitions, and clocks, and may introduce difference constraints in transition guards to choose among invariant copies. Such choices must not create or delete an accepted timed word. The target representation has conjunctive guards and conjunctive location invariants; a later symbolic representation groups transitions with identical source, target, and reset sets under Boolean labels.

All main preservation theorems below are stated for arbitrary input TBAs. Their proof does not require properties of the MTL translation or Spot. An upstream correctness theorem can be composed with these results, but Spot's correctness and the soundness of the front end are outside the generic pass theorems.

III. VERIFIED OPTIMIZATION

After invariant synthesis, the optimization pipeline alternates timing propagation with simplification. Synthesized upper bounds expose how long an automaton can remain at a location. Propagating them to incoming guards can make lower and upper constraints jointly inconsistent; forward propagation supplies complementary lower-bound information. The pipeline then removes transitions, resets, clocks, and duplicate structure that no longer affect the language.

A. INVARIANT SYNTHESIS AND BACKWARD PROPAGATION

For a location ℓ , let $ub_x(g)$ be the largest non-strict upper bound on clock x implied directly by a transition guard g , and $+\infty$ when the guard does not bound x . If outgoing transitions are $\ell \xrightarrow{g_i, Z_i} \ell_i$, the synthesized invariant is

$$Inv(\ell)(x) = \max_i ub_x(g_i),$$

where the maximum of an empty set is $+\infty$. Thus the invariant states the largest upper bound that still permits every outgoing transition. It is non-strict even when its source guard is strict: strictness remains on the transition and need not constrain the interval spent at the location.

Theorem III.1 (Invariant synthesis). *For every TBA $\mathcal{A}$, adding the synthesized invariants preserves its accepted timed words.*

Proof. Removing invariants from a run cannot invalidate a transition. Conversely, on every run, the guard of the transition taken from a location bounds each clock by the synthesized invariant at the event. Monotonicity during the delay

TABLE 1. Verified post-processing passes. Each preservation theorem quantifies over every input TBA under the Coq timed-word semantics. Full definitions and proof arguments appear in the supplementary material.

Pass	Coq preservation theorem
Upper-bound invariant synthesis	add_invariants_accepts
Backward propagation (n steps)	propagate_n_accepts
Forward propagation of lower bounds	forward_accepts
Two-sided invariants	add_two_sided_invariants_accepts
Removal of contradictory transitions	remove_contradictory_accepts
Removal of locations without entering transitions	remove_unreachable_accepts
Removal of dead resets	remove_dead_resets_accepts
Merging synchronously reset clocks	merge_all_accepts
Normalization of guards	normalize_accepts
Removal of unsatisfiable action labels	remove_unsat_accepts
Merging transitions	merge_trans_accepts
Merging equivalent locations	merge_states_all_accepts
Iterated pipeline (n rounds)	optimize_accepts
Merging transitions for export	merge_transitions_accepts
Elimination of disjunctive invariants	split_accepts
Splitting of disjunctive guards	explode_accepts
Normalization of exported guards	normalize_d_accepts
Removal of copies never entered	dprune_n_accepts
Merging of the initial location	dmerge_states_all_accepts
Symbolic transitions	symbolic_ext_accepts
Complete export	export_accepts

gives the invariant throughout that stay. The Coq result is `add_invariants_accepts`; details are in Supplementary Section S1. $\square$

Backward propagation strengthens an edge into ℓ' with the invariant of ℓ' after applying its reset set. A bound on a clock that the edge resets becomes the corresponding constant constraint at zero; an unsatisfiable constant removes that edge. The invariant annotations are then recomputed from the stronger guards.

Theorem III.2 (Backward propagation). *For every TBA $\mathcal{A}$ and every $n \in \mathbb{N}$, the automaton obtained by n backward-propagation steps accepts the same timed words as $\mathcal{A}$.*

Proof. The strengthened guards can only remove runs. Conversely, in an accepting run, the guard on the next transition bounds clock values at the next event by the synthesized invariant of its source. Clock consistency relates these values to the earlier event: a reset clock has value equal to the elapsed delay, and a clock not reset can only increase. Thus the earlier edge satisfies the propagated target invariant after reset substitution. Induction proves preservation for any number of steps. The Coq results are `propagate_accepts` and `propagate_n_accepts`. $\square$

Figure 1 sketches the interaction. A two-location example and all intermediate calculations appear in Supplementary Section S1.

B. FORWARD PROPAGATION AND SIMPLIFICATION

Entry lower bounds are computed per clock and location. A clock reset on every transition entering a location has entry bound zero; otherwise the entry bound is the minimum lower

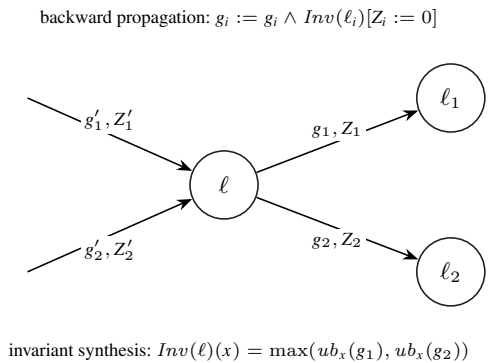


FIGURE 1. Invariant computation. Successor invariants strengthen outgoing guards; upper bounds of the resulting guards resynthesize the source invariant. Reset sets determine which target-clock constraints become constant.

bound guaranteed by every entering transition. If one entry gives no lower bound, no bound is propagated. Since clocks do not decrease during a stay, a valid entry lower bound remains true throughout that location. The forward pass adds these bounds to outgoing guards. A paired representation can carry lower-bound invariants during optimization, but they are an analysis device: UPPAAL does not accept them in emitted location invariants.

After propagation, a guard may contain incompatible upper and lower constraints, for example $x > 5 \wedge x \leq 5$. Such a transition is removed. Edges with contradictory event literals are also removed. Locations with no entering transitions are pruned, with the initial location treated separately. Each removal is language-preserving because the eliminated transition cannot be enabled or its source cannot occur on a run.

Iteration can expose further unreachable locations.

C. DEAD RESETS AND SYNCHRONOUS CLOCK MERGING

A reset is dead when its clock is not tested after the transition on any path that can follow it. The implementation computes a backward closure from locations where the clock is tested. Before removing a reset, the pass checks the two properties used by the proof: no edge leads from a dead location to a live one, and no edge tests the clock in a dead location. Changing reset choices only on these dead portions cannot affect any later guard or acceptance decision. The Coq results are `remove_dead_resets_clock_accepts` and `remove_dead_resets_accepts`.

Two different clocks can be merged if every transition resets them together, no transition enters the initial location, and every edge leaving the initial location resets one without testing it. After the first event, their values are equal; they remain equal thereafter. Every test of the removed clock can therefore be redirected to the retained clock. This condition is sufficient and mechanically checked; the development does not merge clocks merely because sampled values happen to coincide. The theorem is `merge_accepts`, lifted to all clock pairs by `merge_all_accepts`.

D. GUARD NORMALIZATION AND STRUCTURAL REDUCTION

Propagation adds constraints, so guards are tightened by removing any constraint implied by another bound on the same clock. A guard then keeps at most one lower and one upper bound per clock, with equality serving as both. Trivial nonnegative lower bounds are removed using clock nonnegativity. Normalization is repeated after merging because clock renaming can make new constraints redundant.

Transitions with the same source, target, and reset set can be combined when the union of their conditions remains conjunctive. A transition is subsumed when another with the same structural fields has a weaker label and guard. Complementary literals may be resolved, such as p and $\neg p$, $x < 2$ and $x \geq 2$, or $x \leq 5$ and $x > 5$. Equivalent locations are merged only when their Büchi marking agrees and their outgoing transitions have matching twins. That condition prevents merging an accepting location with a non-accepting location and thereby changing which infinite runs are accepting.

E. ITERATED OPTIMIZATION

One round applies, in order, backward propagation, forward propagation, removal of contradictory transitions, removal of unsatisfiable action labels, pruning of locations without incoming transitions, normalization, removal of dead resets, synchronous clock merging, a second normalization, and merging of transitions and locations. The first normalization is needed because forward propagation can add $x \geq 0$ after a reset; without removing this tautology, dead-reset analysis could treat the clock as live. A round can enable simplifications in later rounds.

Theorem III.3 (Optimization). *For every TBA $\mathcal{A}$ and every $n \in \mathbb{N}$, `optimize( $n, \mathcal{A}$ )` accepts exactly the timed words accepted by $\mathcal{A}$.*

Proof. Every constituent operation preserves acceptance, as listed in Table 1. Their composition preserves acceptance, and induction on n proves the statement for every number of rounds. The mechanized result is `optimize_accepts`. $\square$

The extracted function also returns synthesized upper-bound invariants and entry bounds. The tool compares successive automata structurally and stops when they are equal or after 50 rounds. The theorem holds independently of whether iteration reaches a fixed point or uses all 50 rounds. In the tracked benchmark, observed rounds reach a fixed point within five rounds; this is a measured property of those inputs, not a termination assumption in the proof.

IV. UPPAAL-ORIENTED EXPORT

The optimized automaton has normalized conjunctive guards. Export first merges transitions that agree on source, target, event label, and resets, placing their guards into a disjunction. The location invariant is then represented by disjunctions of conjunctions of upper bounds. UPPAAL-oriented output requires each such invariant to be conjunctive.

A location with invariant $I_1 \vee \dots \vee I_r$ is replaced by copies, one for each irredundant conjunctive component I_k . Incoming edges are replicated and strengthened with an entry condition; outgoing edges are replicated from each copy. The entry condition chooses a copy whose invariant admits a delay at least as long as each other copy would admit from the same valuation. This condition compares differences between clocks and is what introduces clock-difference guards in exported models. The initial location is handled by a dedicated copy because it has no incoming transition.

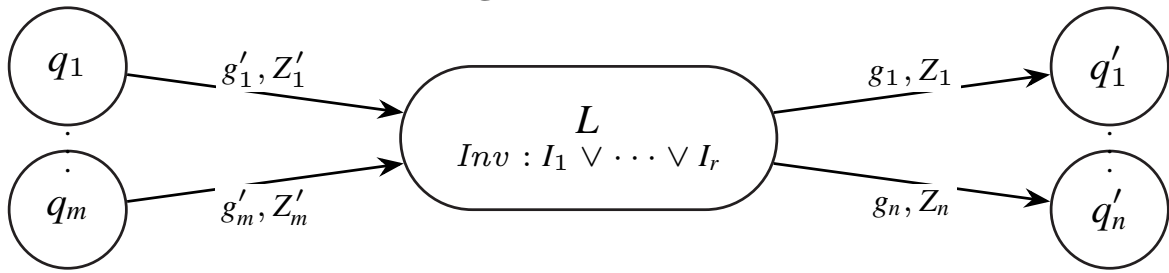
Figure 2 shows the split schematically. Replicating edges without the entry condition would permit a run to choose a copy with a shorter delay, potentially losing words. With the condition, the copies represent the original disjunction while preserving the longest admissible stay.

The export continues by splitting disjunctive guards into conjunctive transitions, normalizing, pruning unreachable copies, and merging the initial location when an equivalent copy exists. A final symbolic pass groups transitions with the same source, target, and resets and stores their labels as Boolean conditions for drawings.

Theorem IV.1 (Export preservation). *For every automaton accepted by the verified export interface, the complete export preserves its timed-word language. The resulting explicit automaton has conjunctive transition guards and conjunctive location invariants. Its symbolic representation also preserves acceptance.*

Proof. Export composes verified transition merging, invariant splitting, guard splitting, normalization, pruning, initial-location merging, and symbolic grouping.

Original automaton



Transformed automaton

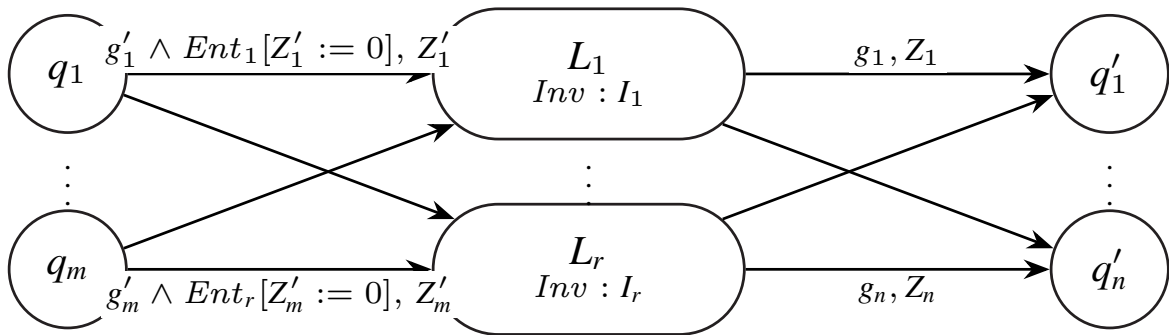


FIGURE 2. Elimination of a disjunctive invariant. Each copy has a conjunctive invariant, and the entry guard selects a copy that admits a maximal delay from the incoming valuation. The entry condition can contain clock-difference constraints.

Each component has a language-preservation theorem, including `split_accepts` for invariant elimination and `export_accepts` for the complete construction. Output shape is established by `export_guards_conjunctive` and `export_invariants_conjunctive`. $\square$

The complete construction and proof of the invariant split, including its entry-condition formula, are in Supplementary Section S1-G. The theorem is generic in the automaton: it requires no MTL provenance, determinism, or particular Spot option.

V. TOOL ARCHITECTURE AND USE IN UPPAAL

A. IMPLEMENTATION AND PROOF BOUNDARY

The OCaml module `optim.ml` is extracted from Coq. It implements the invariant and optimization functions, export passes, initialization check used by the upstream MTL workflow, and symbolic grouping. The surrounding `mtl2tba` executable contains a hand-written parser, calls Spot when building its upstream automaton, reads the returned LBTT automaton, drives the optimization loop, and prints UPPAAL XML and Graphviz DOT. In generic use, the verified functions operate on their Coq TBA type; a parser or converter

for a third-party TBA format is an additional trusted interface unless verified separately.

Pass theorems are independent of the source of their TBA. End-to-end correctness for an MTL formula additionally composes with the upstream translation and its LTL-to-Büchi contract. The Spot call, parser, LBTT reader, driver, and XML/DOT printers are outside the Coq pass proofs. The floating-point realization of extracted real-number operations is also trusted; the tool restricts bounds to integers in $[0, 2^{30}]$, for which inspected arithmetic is exact in the target representation. The repository records Rocq 9.0.1, OCaml 5.4.0, Spot 2.16, and UPPAAL 5.0.0 for the build and case-study artifacts.

The generic theorems use existential initial clock values. UPPAAL starts clocks at zero. The verified `init_free` check and its zero-initialization theorem are part of the MTL-generated workflow; they do not provide an initialization theorem for arbitrary third-party TBAs. Users should check the initialization contract of any external automaton.

B. UPPAAL PROPERTY MODELS

UPPAAL does not check Büchi acceptance directly; accepting locations are retained and named for reachability queries.

For safety requirements, the case studies check whether a violation automaton can reach its accepting sink. When the property automaton is composed with a system, its invariants can constrain time and remove system behaviors. It is therefore necessary to distinguish a violation-reachability product from using an automaton as a passive observer. The supplementary material records the system-side requirements used in our examples: strictly increasing event times, time-divergent executions, synchronization with event channels, and deadlock or timelock checks where relevant.

For liveness, a Büchi emptiness checker must account for accepting runs under divergent time; UPPAAL's ordinary reachability interface does not provide that acceptance condition. The artifact uses TChecker for the liveness example. These use arguments are separate from the Coq language-preservation proofs for each pass.

C. AVAILABILITY

This project contains proof sources, extracted implementation, benchmark tables and raw data, reproduction scripts, and UPPAAL case-study models, together with the LaTeX source and build instructions. The invariant, optimization, and export formalization files contain the generic pass results; their theorem names are listed in Table 1. The project repository is at <https://github.com/efares1/verified-tba-optimization-uppaal>.

VI. EVALUATION

A. QUESTIONS AND SETUP

The evaluation addresses three questions: (1) what structural changes do optimization and export make on the current tool's benchmark automata; (2) how does the pipeline scale on conjunctions of bounded responses and nested bounded eventualities; and (3) do the output models support the included UPPAAL and timed-language checks? These are artifact and benchmark questions. The correctness theorems are universal in the input TBA, but measured inputs come from the current MTL-to-TBA workflow.

The tracked benchmark contains 31 configurations: requirement patterns F1–F14 and F3b; Rn , which conjoins n bounded responses for $1 \leq n \leq 8$; and Nn , a nested bounded-eventuality family for $1 \leq n \leq 8$. Each chain was run five times with a 300-second limit. Total times are medians; stage-level timing, min/max, memory, exact formulas, and all rows are in Supplementary Section S2. Runs used Ubuntu 26.04 under WSL 2 on an Intel Core i7-8650U at 1.9 GHz with 11.7 GB memory, Rocq/Coq 9.0.1, OCaml 5.4.0, and Spot 2.16. UPPAAL case studies used version 5.0.0. CASAAL ran natively on Windows on the same machine, so timing comparisons across operating systems are indicative only.

Table 2 selects five tracked configurations to show the trade-off between optimization and export. Every number is copied from the committed evaluation table; the full 31-row table is in the supplement.

TABLE 2. Representative tracked benchmark rows. Spot gives upstream states/transitions; Opt. and Export give locations/transitions/clocks; Diff. counts clock-difference constraint occurrences in exported guards; Sym. counts symbolic transitions. Time is median end-to-end seconds over five runs.

ID	Spot (st/tr)	Opt. (loc/tr/clock)	Export (loc/tr/clock)	Diff.	Sym.	Time
F11	5/11	4/7/1	4/7/1	0	7	0.03
R4	16/625	16/112/4	16/217/4	192	80	0.41
R6	64/15625	64/640/6	64/1837/6	3360	448	36.75
N4	6/36	6/25/4	6/25/4	0	11	0.06
N8	10/166	10/81/8	10/81/8	0	19	103.02

B. STRUCTURAL EFFECT OF THE PASSES

The post-processing reduces transitions by 96% in R6 (15,625 to 640), 82% in R4 (625 to 112), and 53% in F13 (17 to 8); R5 falls from 3,125 to 272. These compare the upstream automaton with the output of the post-processing chain, which includes reset completion in this tool. They illustrate the observed benefit on generated inputs, not a reduction bound for arbitrary TBAs.

Export can reverse part of that reduction because it expands disjunctive guards into explicit conjunctive edges and adds entry conditions to invariant copies. R4 grows from 112 optimized edges to 217 exported edges, with 192 difference-constraint occurrences. R6 grows from 640 to 1,837 edges, with 3,360 such occurrences. On N4, 25 explicit edges become 11 symbolic transitions. The number of locations does not change in these examples. Thus export trades compact disjunctions for an explicit conjunctive representation suitable for UPPAAL-oriented output.

The pipeline removes one of two synchronously reset clocks in F11. Other clock-count changes in the full table can arise upstream: for example, the default formula rewrite changes F8 before the generic optimizer sees the automaton. We do not attribute such changes to generic optimization unless a pass is responsible.

C. SCALING AND BACK-END CHOICES

For the bounded-response family, R6 completes in 36.75 seconds, of which optimization takes 16.57 seconds; R7 and R8 exceed the 300-second limit. The nested family reaches N8 in 103.02 seconds and 961 MB, with Spot taking 101.42 seconds. Optimization takes 0.24 seconds on N8; most time is in the upstream Spot construction. Spot's transition-cube representation grows rapidly in Rn , and every later pass processes it. Sharing Boolean subformulas or retaining Boolean labels earlier are possible changes, but their effect was not measured.

The pipeline does not require a deterministic TBA. Spot's default small-automaton option is used for the tracked table; its determinism-preference option grows N4 from 6 locations and 25 optimized transitions to 34 locations and 1,035 transitions and times out on larger nested instances. This is a back-end configuration ablation, not a proof condition. Full

stage timings, memory, and weak-until measurements are in Supplementary Tables S3 and S4.

D. LANGUAGE CHECKS AND COMPARISON LIMITS

The tracked evaluation cross-checks the Spot output, the parser/printer interface, and products with CASAAL using TChecker. Of the 29 configurations with comparable formulas, language inclusions are established on exported automata for 17 and on pre-optimization compiled automata for three; nine cases have no conclusion within limits. The remaining two configurations use hatted operators for which CASAAL has no equivalent operator, so their comparison is not an equivalence claim. Equal sizes alone do not establish language equivalence.

The Spot cross-check succeeds on 25 of 31 configurations; ltl2ba does not finish within 300 seconds on R6–R8 and N6–N8. The interface check passes on all 31 configurations. The implementation simulator passes 18 strict/non-strict boundary and overlapping-activation checks. These checks concern the integrated implementation and trusted interfaces; they supplement but do not replace the Coq proofs.

E. UPPAAL CASE STUDIES

The supplement describes two small systems: a sporadic event emitter and a bounded-response controller, each with a correct and a faulty parameterization. With UPPAAL 5.0.0, the violation query $E \leftrightarrow P.A4$ is false for both correct systems and true for both faulty systems. Diagnostic traces are $(e, 1)(e, 2)$ for the sporadicity violation and $(p, 0.5)(q, 6)$ for the response violation. The system-alone query $A[] \text{ not deadlock}$ holds for the four packaged models; a first controller variant had a timelock and was corrected. This demonstrates a workflow, not an industrial evaluation.

F. THREATS TO VALIDITY

The benchmark is fixed, hand-selected, and generated through one MTL-based toolchain; it contains no industrial specification and is not a broad arbitrary-TBA suite. R7/R8 time out, and output sizes are sensitive to Spot version and options. The CASAAL language comparison is limited by operator coverage, common-domain encoding, product-checker limits, and an unverified conversion script. The tools ran under different operating systems. The case study consists of four small parameterized models. These limitations concern empirical generality; they do not narrow the universal quantification of the Coq preservation theorems.

VII. RELATED WORK

The work relates to semantics-preserving simplification of timed automata, disjunctive invariants in UPPAAL, and mechanically verified automata software.

Bodeveix and Filali studied disjunctive invariants in UPPAAL and their elimination by automaton restructuring [1]; Bodeveix, Graf, and Filali considered verification with timed automata and invariant handling [2]. Our export uses location copies and entry conditions that select a copy with a maximal

admissible delay, and proves the split preserves timed-word language. Optimization also covers bound propagation, reset removal, and merging for arbitrary TBA inputs.

Timed automata and their language semantics have a broad verification literature; standard treatments include timed Büchi automata and timed words [3]. UPPAAL and TChecker provide analysis engines, while this work addresses a verified normalization and export layer before analysis. TChecker is used as an external emptiness checker for selected comparisons and a liveness task, not as part of the Coq proof [4].

Mechanized automata verification includes verified LTL-to-Büchi translation and model checking in Isabelle/HOL, including CAVA [5], [6], and verified timed-automata model checking in Munta [7], [8]. Those tools address upstream translation or downstream checking. Our transformations can be composed with a correct upstream construction; a verified checker can consume the exported automaton, provided a representation and proof bridge is supplied. The extraction mechanism for the OCaml implementation is described by Letouzey [9].

VIII. CONCLUSION

We presented a Coq-verified optimization and export pipeline for timed Büchi automata. The transformations synthesize and propagate location invariants, simplify timing constraints, remove infeasible edges and dead resets, merge synchronously reset clocks and equivalent automaton structure, and eliminate disjunctive invariants to produce conjunctive guards and invariants. The optimization theorem is generic in its TBA input and holds for every number of rounds; export preserves the same language and has verified output-shape properties.

The extracted implementation is used by mtl2tba. Tracked measurements show substantial transition reductions on several generated examples, while also exposing the cost of expanding disjunctions for explicit UPPAAL output and the scaling limits of the upstream automata. Four UPPAAL case-study models produce the expected safety-query verdicts. These measurements characterize the existing benchmark and tool chain; they do not establish performance over a broad collection of arbitrary TBAs.

Future work includes evaluating the generic passes on independent TBA corpora, improving symbolic handling of Boolean transition labels before Spot expansion, extending clock merging beyond synchronous reset, and verifying additional input/output adapters so third-party formats can use the pipeline with smaller trusted interfaces.

REFERENCES

- [1] J.-P. Bodeveix and M. Filali, “About disjunctive invariants in UPPAAL (NWPT, Turku Finlande, 29/10/03–31/10/03),” in *NWPT, Turku Finlande, 29/10/03–31/10/03*, octobre 2003. [Online]. Available: <http://www.abo.fi/~Kaisa.Sere/15NWPT.html>
- [2] G. Behrmann, A. David, and K. G. Larsen, “A tutorial on UPPAAL,” in *Formal Methods for the Design of Real-Time Systems*, ser. Lecture Notes in Computer Science. Springer, 2004, vol. 3185, pp. 200–236.

- [3] S. Tripakis, “Checking timed Büchi automata emptiness on simulation graphs,” *ACM Trans. Comput. Logic*, vol. 10, no. 3, Apr. 2009. [Online]. Available: <https://doi.org/10.1145/1507244.1507245>
- [4] F. Herbreteau and G. Point, “TChecker: model checker for timed automata,” <https://github.com/ticktac-project/tchecker>, 2025, version 0.8.
- [5] A. Schimpf, S. Merz, and J.-G. Smaus, “Construction of Büchi automata for LTL model checking verified in Isabelle/HOL,” in *Theorem Proving in Higher Order Logics (TPHOLs 2009)*, ser. Lecture Notes in Computer Science, vol. 5674. Springer, 2009, pp. 424–439.
- [6] J. Esparza, P. Lammich, R. Neumann, T. Nipkow, A. Schimpf, and J.-G. Smaus, “A fully verified executable LTL model checker,” in *Computer Aided Verification (CAV 2013)*, ser. Lecture Notes in Computer Science, vol. 8044. Springer, 2013, pp. 463–478.
- [7] S. Wimmer and P. Lammich, “Verified model checking of timed automata,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2018)*, ser. Lecture Notes in Computer Science. Springer, 2018, pp. 61–78.
- [8] S. Wimmer, “Munta: A verified model checker for timed automata,” in *Formal Modeling and Analysis of Timed Systems (FORMATS 2019)*, ser. Lecture Notes in Computer Science. Springer, 2019, pp. 236–243.
- [9] P. Letouzey, “Extraction in Coq: An overview,” in *Logic and Theory of Algorithms, Fourth Conference on Computability in Europe (CiE 2008)*, ser. Lecture Notes in Computer Science, vol. 5028. Springer, 2008, pp. 359–369.

...